DAY 2

Python for Data, APIs & Automation

Turn Python into a practical data and automation engine

What you will build

An AI Student Analytics Dashboard | A messy-data cleaning pipeline | API and file automation scripts

Explanation + Code + Output walkthroughs + Analogies + Flow diagrams + Practice challenges

AI & Data Science Workbook | Python 3 | pandas, NumPy, Matplotlib, requests

	Section	What it covers	Page
1	**Advanced Python**	Comprehensions, map(), filter(), zip(), enumerate()	3
2	**File Handling**	TXT, CSV, JSON and Python dictionaries	6
3	**NumPy**	Arrays and mathematical operations	11
4	**Pandas**	DataFrames, CSV loading, filter, sort, group, missing data	13
5	**Data Visualization**	Matplotlib: bar, line, pie and scatter	18
6	**APIs with Python**	GET, POST, JSON, status codes, API keys	21
7	**Automation**	Files, CSV, cleaning, APIs and reports	26
8	**Hands-On Projects**	AI Student Analytics Dashboard and Messy Data Challenge	30

	Section	What it covers	Page
9	**Reference**	Cheat sheet, self-check quiz and answer key	35

How to use this workbook

- **Type every example yourself.** Reading code feels like understanding; typing it is what makes it stick.
- **All outputs shown were produced by running the code.** If your output differs, compare line by line: it is almost always a typo or a different file path.
- **Each topic has the same rhythm:** idea, analogy, code, output walkthrough, flow diagram, then challenges.
- **Attempt the challenges before looking at hints.** Answers to the key ones are in Section 9.

Setup (5 minutes)

CODE

```
# 1. Create a folder and a virtual environment
mkdir day2 && cd day2
python -m venv venv
source venv/bin/activate            # Windows: venv\Scripts\activate

# 2. Install the libraries used in this workbook
pip install numpy pandas matplotlib requests python-dotenv

# 3. Check
python -c "import pandas, numpy, matplotlib, requests; print('ready')"
```

> **PRO TIP**
> Use VS Code or Jupyter Notebook. For each code block, create a new cell or file. Files created by the examples (notes.txt, students.csv ...) appear in your current working folder.

The five-step data workflow used all day



Every section below is one stage of this pipeline. The two projects at the end connect all five.

SECTION 1

Advanced Python

Write shorter, faster, more readable code for data work

1.1 List and dictionary comprehensions

A **comprehension** builds a new collection from an existing one in a single expression. Instead of creating an empty list, looping, and appending, you describe the result you want. The general pattern is `[expression for item in iterable if condition]`. The same idea works for dictionaries `{k: v for ...}` and sets `{x for ...}`.

REAL-WORLD ANALOGY

A factory conveyor belt. Raw items roll in (*iterable*). A quality inspector removes bad ones (*if condition*). A stamping machine reshapes the rest (*expression*). Everything that survives is boxed at the end (*the new list*).



Flow of a list comprehension: items failing the condition never reach the expression

Lists: filter and transform

CODE

```
marks = [72, 45, 88, 91, 33, 67]

# Classic loop
passed_loop = []
for m in marks:
    if m >= 40:
        passed_loop.append(m)

# Same thing as a comprehension
passed  = [m for m in marks if m >= 40]
bonus   = [m + 5 if m < 50 else m for m in marks]   # if/else goes BEFORE 'for'
squares = [n ** 2 for n in range(1, 6)]

print("loop    :", passed_loop)
print("passed  :", passed)
print("bonus   :", bonus)
print("squares :", squares)
```

OUTPUT

```
loop    : [72, 45, 88, 91, 67]
passed  : [72, 45, 88, 91, 67]
bonus   : [72, 50, 88, 91, 38, 67]
squares : [1, 4, 9, 16, 25]
```

OUTPUT WALKTHROUGH

- **passed** keeps only marks that are 40 or more, so 33 is dropped. The order of the survivors is unchanged.
- **bonus** keeps every item (no filter) but transforms some: 45 becomes 50 and 33 becomes 38.
- **Position rule:** a lone `if` (filter) goes at the *end*; an `if ... else` (choose a value) goes at the *start*.

Dictionaries and sets

CODE

```
names  = ["Asha", "Ravi", "Meena", "Karan"]
scores = [88, 72, 95, 45]

report  = {n: s for n, s in zip(names, scores)}
toppers = {n: s for n, s in report.items() if s >= 80}
grade   = {n: ("A" if s >= 85 else "B" if s >= 60 else "C") for n, s in report.items()}
name_lengths = {len(n) for n in names}     # a set keeps unique values only

print(report)
print(toppers)
print(grade)
print(name_lengths)
```

OUTPUT

```
{'Asha': 88, 'Ravi': 72, 'Meena': 95, 'Karan': 45}
{'Asha': 88, 'Meena': 95}
{'Asha': 'A', 'Ravi': 'B', 'Meena': 'A', 'Karan': 'C'}
{4, 5}
```

OUTPUT WALKTHROUGH

report pairs each name with its score. **toppers** filters by value using `.items()`. **grade** uses a chained conditional expression. **name_lengths** shows {4, 5} because the set removes the duplicates (Asha and Ravi are both 4 letters, Meena and Karan both 5).

Nested comprehensions

CODE

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flat = [x for row in matrix for x in row]
transposed = [[row[i] for row in matrix] for i in range(3)]
print(flat)
print(transposed)
```

OUTPUT

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
[[1, 4, 7], [2, 5, 8], [3, 6, 9]]
```

PRO TIP

Read nested comprehensions in the same order as nested loops: `for row in matrix` runs first, then `for x in row`. If a comprehension needs more than two *for* clauses or does not fit on one screen line comfortably, switch back to a normal loop. Readability wins.

WATCH OUT

Do not use a comprehension only for its side effects, for example `[print(x) for x in data]`. It builds a throwaway list. Use a plain `for` loop when you are not creating a collection.

PRACTICE CHALLENGES

1. From `["data", "AI", "python", "ML", "pandas"]` build a list of the **lengths** of words longer than 2 letters. *Hint: put the filter on len(w).*

2. Create a dict mapping each number 1 to 10 to `True` if even, else `False`. *Hint: n % 2 == 0.*

3. From a list of emails build a **set** of unique domains. *Hint: email.split('@')[1].*

4. Turn `[[1, 2], [3, 4], [5, 6]]` into a flat list of squares. *Hint: nested for, square the x.*

1.2 map(), filter(), zip() and enumerate()

These four built-ins are the everyday tools for looping over data without writing index bookkeeping. They return **lazy iterators**: nothing is computed until you loop over them or wrap them in `list()`.

Function	What it does	Returns	Mental model
`map(f, xs)`	Applies f to every item	iterator of results	stamp every item
`filter(f, xs)`	Keeps items where f(item) is True	iterator of survivors	security check at a gate
`zip(a, b)`	Pairs items at the same position	iterator of tuples	zip fastener joining two rows
`enumerate(xs)`	Adds a counter to each item	iterator of (index, item)	numbering the seats

REAL-WORLD ANALOGY

Think of a school exam hall. **map** is a teacher applying the same curve to every paper. **filter** is the invigilator letting in only students with a hall ticket. **zip** matches roll numbers to names from two separate lists. **enumerate** stamps a seat number on each student as they walk in.



A small pipeline: each built-in does one job and hands its result to the next

map() and filter()

CODE

```
prices = [100, 250, 80, 400]
with_gst = list(map(lambda p: round(p * 1.18, 2), prices))
costly   = list(filter(lambda p: p > 90, prices))
lazy     = map(str.upper, ["a", "b"])

print(with_gst)
print(costly)
print(type(lazy))
print(list(lazy))
```

OUTPUT

```
[118.0, 295.0, 94.4, 472.0]
[100, 250, 400]
<class 'map'>
['A', 'B']
```

OUTPUT WALKTHROUGH

lambda is a tiny unnamed function: `lambda p: p * 1.18` means 'given p, return p times 1.18'. The **type** line proves map() gave back a lazy `map` object, not a list. Only `list(lazy)` actually runs it. Note that a comprehension can do both jobs: `[p for p in prices if p > 90]`.

zip() and enumerate() together

CODE

```
names  = ["Asha", "Ravi", "Meena"]
maths  = [88, 72, 95]
python = [92, 65, 98]

for rank, (n, m, p) in enumerate(zip(names, maths, python), start=1):
    print(f"{rank}. {n:<6} maths={m} python={p} avg={(m + p) / 2}")

pairs = list(zip(names, maths))
n2, m2 = zip(*pairs)                    # 'unzip'
best   = max(zip(maths, names))         # compares maths first
ranked = sorted(zip(names, maths), key=lambda t: t[1], reverse=True)
print(pairs)
print(n2, m2)
print(best)
print(ranked)
```

OUTPUT

```
1. Asha   maths=88 python=92 avg=90.0
2. Ravi   maths=72 python=65 avg=68.5
3. Meena  maths=95 python=98 avg=96.5
[('Asha', 88), ('Ravi', 72), ('Meena', 95)]
('Asha', 'Ravi', 'Meena') (88, 72, 95)
(95, 'Meena')
[('Meena', 95), ('Asha', 88), ('Ravi', 72)]
```

OUTPUT WALKTHROUGH

- `enumerate(..., start=1)` gives the rank counter 1, 2, 3 instead of 0, 1, 2.
- The loop header `rank, (n, m, p)` **unpacks** the nested tuple that zip produces.
- `zip(*pairs)` reverses a zip: the star spreads the pairs out as separate arguments.
- `max(zip(maths, names))` works because tuples compare left to right, so the highest maths mark wins and the name rides along.

WATCH OUT

`zip` stops at the **shortest** list without warning. If your lists must be the same length, check `len()` first or use `zip(a, b, strict=True)` (Python 3.10+) to get an error instead of silent data loss.

PRACTICE CHALLENGES

1. Given temperatures in Celsius `[0, 25, 37, 100]` use **map** to convert to Fahrenheit (F = C x 9/5 + 32). *Hint: lambda c: c * 9 / 5 + 32.*

2. Use **filter** to keep only the names that start with a vowel. *Hint: n[0].lower() in 'aeiou'.*

3. Use **zip** to build a dictionary from two lists: subjects and marks. *Hint: dict(zip(subjects, marks)).*

4. Print a numbered list of subjects starting at 1 using **enumerate**. *Hint: start=1.*

5. Boss level: combine zip, filter and sorted to list students scoring above the average, best first. *Hint: compute avg = sum(m)/len(m) first.*

SECTION 2

File Handling

Read and write TXT, CSV and JSON, the three formats data lives in

Almost every data project starts with a file and ends with a file. Python can open all three of the most common formats with the standard library alone. You will use `open()` for text, the `csv` module for tables and the `json` module for nested data.

Format	Looks like	Best for	Python tool
TXT	plain lines of text	logs, notes, simple lists	`open()`, read, write

Format	Looks like	Best for	Python tool
CSV	rows and columns separated by commas	tables, spreadsheets, exports	`csv` module, pandas
JSON	nested keys and values	APIs, configs, hierarchical data	`json` module

2.1 Reading and writing TXT files

REAL-WORLD ANALOGY

A file is a notebook. Mode `"r"` means you may only read it. Mode `"w"` tears out every page and starts fresh, so be careful. Mode `"a"` writes on the next empty line at the end. The `with` statement is a helpful assistant who always closes the notebook for you, even if something goes wrong.



Life cycle of a file operation

Mode	Meaning	If file exists	If file is missing
`"r"`	read (default)	opens it	FileNotFoundError
`"w"`	write	**erases** it first	creates it
`"a"`	append	adds to the end	creates it
`"r+"`	read and write	opens it	FileNotFoundError

CODE

```
with open("notes.txt", "w") as f:
    f.write("Day 2: Python for data\n")
    f.write("Files are just streams of text\n")

with open("notes.txt", "a") as f:
    f.write("Appended line\n")

with open("notes.txt") as f:
    for number, line in enumerate(f, start=1):
        print(number, line.strip())

with open("notes.txt") as f:
    words = f.read().split()
print(len(words), "words; longest:", max(words, key=len))
```

OUTPUT

```
1 Day 2: Python for data
2 Files are just streams of text
3 Appended line
13 words; longest: Appended
```

OUTPUT WALKTHROUGH

- First block: mode `"w"` creates the file with two lines. `\n` is the newline character, since write() never adds one for you.
- Second block: mode `"a"` adds a third line without touching the first two.
- Third block: iterating over the file object gives one line at a time (memory friendly for huge files). `.strip()` removes the trailing newline so print does not double-space.
- Fourth block: `.read().split()` loads everything and splits on whitespace: 13 words, and `max(words, key=len)` finds the longest one.

WATCH OUT

Opening with `"w"` on an existing file deletes its contents immediately. When in doubt, write to a new filename first or use `"a"`. Also catch `FileNotFoundError` when reading files supplied by users.

PRACTICE CHALLENGES

1. Write ten numbers, one per line, to `numbers.txt`, then read them back and print their sum. *Hint: int(line) for each line.*

2. Count how many lines of a text file contain the word 'python' (case-insensitive). *Hint: 'python' in line.lower().*

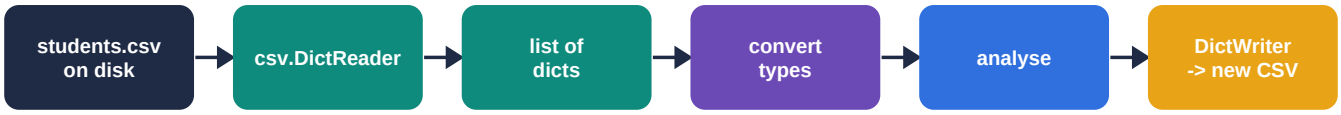
3. Write a function `safe_read(path)` that returns an empty string when the file does not exist. *Hint: try / except FileNotFoundError.*

2.2 CSV processing

CSV (comma separated values) is a table in text form. The first line normally holds column names. The `csv` module handles awkward details such as commas inside quoted values. **DictReader** turns every row into a dictionary keyed by the header, which is far more readable than numeric positions.

REAL-WORLD ANALOGY

A CSV file is a spreadsheet with all the formatting stripped away. Each line is a row and each comma is the wall between two cells. Opening it in Excel simply draws those walls for you.



CSV round trip: read, convert, analyse, write

Create the practice file

CODE

```
import csv

rows = [
    ["name", "section", "maths", "python", "dbms", "attendance"],
    ["Asha", "A", 88, 92, 79, 94],
    ["Ravi", "B", 72, 65, 70, 81],
    ["Meena", "A", 95, 98, 91, 97],
    ["Karan", "C", 45, 52, "", 68],
    ["Divya", "B", 67, "", 74, 88],
    ["Imran", "C", 33, 41, 38, 59],
    ["Sneha", "A", 81, 77, 85, 90],
    ["Tarun", "B", 58, 61, 55, 76],
]
with open("students.csv", "w", newline="") as f:
    csv.writer(f).writerows(rows)
print("rows written:", len(rows) - 1)
```

OUTPUT

```
rows written: 8
```

PRO TIP

Two cells in this file are deliberately **empty** (Karan's dbms and Divya's python). Real data always has gaps. We will detect and repair them with pandas later in Section 4.

Read, convert and analyse

CODE

```
with open("students.csv", newline="") as f:
    students = list(csv.DictReader(f))

from pprint import pprint
pprint(students[0], width=50, sort_dicts=False)
print(type(students[0]["maths"]))

maths = [int(s["maths"]) for s in students]
dbms  = [int(s["dbms"]) for s in students if s["dbms"] != ""]
print("avg maths:", sum(maths) / len(maths))
print("dbms marks available:", len(dbms), "of", len(students))
```

OUTPUT

```
{'name': 'Asha',
 'section': 'A',
 'maths': '88',
 'python': '92',
 'dbms': '79',
 'attendance': '94'}
<class 'str'>
avg maths: 67.375
dbms marks available: 7 of 8
```

OUTPUT WALKTHROUGH

- Row 0 prints as a dictionary. Every value is a **string**, including '88'. The next line confirms it: `<class 'str'>`.
- You must convert with `int()` or `float()` before doing arithmetic.
- The dbms comprehension skips empty strings using an `if` filter; otherwise `int("")` would crash. Only 7 of 8 values survive.

Filter and write a new CSV

CODE

```
toppers = [s for s in students if int(s["maths"]) >= 80]
with open("toppers.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=students[0].keys())
    writer.writeheader()
    writer.writerows(toppers)

with open("toppers.csv") as f:
    print(f.read())
```

OUTPUT

```
name,section,maths,python,dbms,attendance
Asha,A,88,92,79,94
Meena,A,95,98,91,97
Sneha,A,81,77,85,90
```

WATCH OUT

Always pass `newline=""` when opening CSV files for the csv module. Without it Windows may insert blank lines between rows.

PRACTICE CHALLENGES

1. Print the name of the student with the highest python mark. Skip empty values. *Hint: filter empty strings, then max(..., key=...).*

2. Write `section_a.csv` containing only section A students. *Hint: DictWriter with a filtered list.*

3. Add a new column `total` (maths + python) and save as `with_total.csv`. *Hint: remember to add 'total' to fieldnames.*

2.3 JSON handling and Python dictionaries

JSON (JavaScript Object Notation) is the universal language of web APIs. It looks almost exactly like a Python dictionary, and that is no accident: a JSON *object* maps to a Python `dict`, a JSON *array* maps to a `list`. Once you are fluent with nested dictionaries you are already fluent with JSON.

REAL-WORLD ANALOGY

JSON is a form with labelled fields. Some fields hold a simple answer (name: Asha). Some hold a sub-form (marks: another set of labelled fields). Some hold a list of things (skills). You follow the labels to reach any answer.

Python	JSON	Example
`dict`	object	{"name": "Asha"}
`list`	array	[1, 2, 3]
`str`	string	"hello"

Python	JSON	Example
`int / float`	number	42, 3.14
`True / False`	true / false	true, false (lowercase)
`None`	null	null

Nested dictionary basics

CODE

```
import json
student = {
    "name": "Asha", "section": "A",
    "marks": {"maths": 88, "python": 92, "dbms": 79},
    "skills": ["Python", "SQL"],
    "placed": False, "mentor": None,
}
print(student["marks"]["python"])
print(student.get("hostel", "not provided"))
student["marks"]["ml"] = 90
for subject, mark in student["marks"].items():
    print(subject, mark)
```

OUTPUT

```
92
not provided
maths 88
python 92
dbms 79
ml 90
```

> **PRO TIP**
> Use `d.get(key, default)` when a key might be missing. `d[key]` raises a **KeyError** instead, which is what you want only when a missing key is a genuine bug.

Python to JSON text and back

CODE

```
text = json.dumps(student, indent=2)
print(text)

with open("student.json", "w") as f:
    json.dump(student, f, indent=2)
with open("student.json") as f:
    loaded = json.load(f)
print(loaded == student, type(loaded["placed"]))
```

OUTPUT

```
{
  "name": "Asha",
  "section": "A",
  "marks": {
    "maths": 88,
    "python": 92,
    "dbms": 79,
    "ml": 90
  },
  "skills": [
    "Python",
    "SQL"
  ],
  "placed": false,
  "mentor": null
}
True <class 'bool'>
```

OUTPUT WALKTHROUGH

Notice that `False` became `false` and `None` became `null` in the JSON text, then converted back losslessly: the last line prints `True` for equality and a real Python `bool`. Remember the mnemonic: `dump/load` work with **files**, `dumps/loads` work with **strings** (the 's').

JSON data analysis

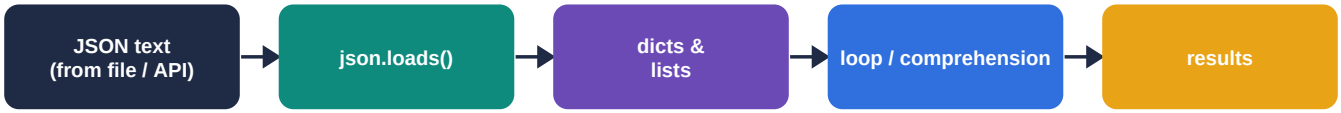
CODE

```
records = json.loads('''[
  {"name": "Asha",  "marks": {"maths": 88, "python": 92}},
  {"name": "Ravi",  "marks": {"maths": 72, "python": 65}},
  {"name": "Meena", "marks": {"maths": 95, "python": 98}}
]''')

avg  = {r["name"]: sum(r["marks"].values()) / len(r["marks"]) for r in records}
best = max(avg, key=avg.get)
print(avg)
print("Top:", best)
```

OUTPUT

```
{'Asha': 90.0, 'Ravi': 68.5, 'Meena': 96.5}
Top: Meena
```



Analysing JSON is just working with dictionaries and lists

WATCH OUT

JSON keys are always strings, so `{1: 'a'}` comes back as `{"1": "a"}`. Tuples become lists. Sets and custom objects cannot be dumped directly.

PRACTICE CHALLENGES

1. Create a dict for a book (title, author, tags list, nested publisher dict). Save it as `book.json` and reload it. *Hint: json.dump with indent=2.*

2. Given a list of JSON records, count how many students have a python mark above 70. *Hint: sum(1 for r in records if ...).*

3. Write a function `get_nested(d, *keys, default=None)` that safely walks nested keys. *Hint: loop over keys with d = d.get(k, {}).*

SECTION 3

NumPy

Fast arrays and mathematical operations without writing loops

3.1 Arrays and mathematical operations

NumPy provides the `ndarray`: a grid of numbers that all share one type. Because the data is stored compactly and the loops run in compiled code, NumPy is typically far faster than Python lists for numeric work. It is also the foundation under pandas, scikit-learn and most of the AI stack.

REAL-WORLD ANALOGY

A Python list is a shopping bag of mixed items: flexible, but you handle each item one at a time. A NumPy array is an egg tray: every slot is the same kind of thing, so you can lift, stamp or inspect the whole tray in one move. In Excel terms, it is dragging one formula down an entire column.



Vectorization: one expression acts on every element, no for loop needed

Creating arrays and element-wise math

CODE

```
import numpy as np

marks = np.array([72, 45, 88, 91, 33, 67])
print(marks, marks.dtype, marks.shape)

print([1, 2, 3] * 2)          # list: repeats
print(np.array([1, 2, 3]) * 2)  # array: multiplies

print(marks + 5)
print(marks / 100)
print("mean", marks.mean(), "median", np.median(marks), "std", round(marks.std(), 2))
print("best index", marks.argmax(), "value", marks.max())
```

OUTPUT

```
[72 45 88 91 33 67] int64 (6,)
[1, 2, 3, 1, 2, 3]
[2 4 6]
[77 50 93 96 38 72]
[0.72 0.45 0.88 0.91 0.33 0.67]
mean 66.0 median 69.5 std 21.12
best index 3 value 91
```

OUTPUT WALKTHROUGH

- `dtype int64` and `shape (6,)` tell you the element type and that this is a one-dimensional array of 6 values.
- The list times 2 **repeats** the list; the array times 2 **multiplies each value**. This difference surprises everyone once.
- Median is the middle of the sorted values (33, 45, 67, 72, 88, 91), so it is the average of 67 and 72, which is 69.5.
- `argmax()` returns the *position* of the largest value (3), while `max()` returns the value itself (91).

Two-dimensional arrays and the axis argument

CODE

```
scores = np.array([[88, 92, 79],
                   [72, 65, 70],
                   [95, 98, 91],
                   [45, 52, 40]])      # 4 students x 3 subjects

print(scores.shape)
print("per student :", scores.mean(axis=1).round(1))
print("per subject :", scores.mean(axis=0).round(1))
print("overall     :", scores.mean().round(2))
```

OUTPUT

```
(4, 3)
per student : [86.3 69.  94.7 45.7]
per subject : [75.  76.8 70. ]
overall     : 73.92
```

KEY IDEA

axis=0 means 'collapse the rows', so you get one answer per **column** (per subject). **axis=1** means 'collapse the columns', so you get one answer per **row** (per student). A helpful trick: the axis you name is the one that disappears from the shape.

Boolean masks, np.where and broadcasting

CODE

```
print(marks[marks >= 40])                       # boolean mask
print("failed:", (marks < 40).sum())            # True counts as 1
print(np.where(marks >= 40, "Pass", "Fail"))    # vectorized if/else
print(np.clip(marks + 10, 0, 100))              # cap at 100

weights  = np.array([0.5, 0.3, 0.2])            # maths, python, dbms
weighted = (scores * weights).sum(axis=1)       # broadcasting across rows
print(weighted.round(1))
```

OUTPUT

```
[72 45 88 91 67]
failed: 1
['Pass' 'Pass' 'Pass' 'Pass' 'Fail' 'Pass']
[ 82  55  98 100  43  77]
[87.4 69.5 95.1 46.1]
```

OUTPUT WALKTHROUGH

- **Mask:** `marks >= 40` creates six True/False values; using it as an index keeps only the True positions (33 disappears).
- **Counting:** summing booleans counts the Trues, giving 1 failure.
- **Broadcasting:** the 1x3 weight vector is stretched across all 4 rows automatically, so each student's marks are multiplied subject by subject, then summed into a weighted score.

Task	Code	Task	Code
Sum / mean	`a.sum()`, `a.mean()`	Sort	`np.sort(a)`
Min / max index	`a.argmin()`, `a.argmax()`	Percentile	`np.percentile(a, 90)`
Create range	`np.arange(0, 10, 2)`	Evenly spaced	`np.linspace(0, 1, 5)`
Zeros / ones	`np.zeros((2, 3))`	Reshape	`a.reshape(2, 3)`

WATCH OUT

NumPy arrays hold **one type**. Mixing numbers and strings silently turns everything into text. Also, `a == b` on arrays gives an array of booleans; use `np.array_equal(a, b)` to compare whole arrays.

PRACTICE CHALLENGES

1. Create `np.arange(1, 21)` and print only the numbers divisible by 3. *Hint: a[a % 3 == 0].*

2. Convert a 1-D array of Celsius temperatures to Fahrenheit with a single expression. *Hint: a * 9 / 5 + 32.*

3. For the 4x3 `scores` array find the subject (column) with the highest average and the student (row) with the lowest. *Hint: argmax on mean(axis=0), argmin on mean(axis=1).*

4. Standardise `marks` to z-scores: (x - mean) / std. *Hint: no loop needed.*

SECTION 4

Pandas

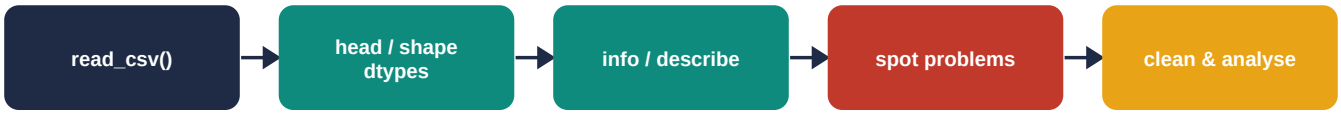
The spreadsheet engine of Python: load, explore, filter, group and repair data

4.1 DataFrames and CSV loading

A **DataFrame** is a table with labelled rows and columns. Each column is a **Series**: a NumPy array with an index. You will spend most of your data career inside DataFrames, so learn to inspect one with a fixed routine: `head`, `shape`, `dtypes`, `info`, `describe`.

REAL-WORLD ANALOGY

A DataFrame is an Excel sheet with superpowers. Columns have names, each column enforces a data type, and a formula you write once applies to every row instantly. Unlike Excel, everything you do is recorded in code, so you can repeat it on next month's file with one click.



First-look routine for every new dataset

CODE

```
import pandas as pd

df = pd.read_csv("students.csv")
print(df)
print(df.shape)
```

OUTPUT

```
    name section  maths  python  dbms  attendance
0   Asha       A     88    92.0  79.0          94
1   Ravi       B     72    65.0  70.0          81
2  Meena       A     95    98.0  91.0          97
3  Karan       C     45    52.0   NaN          68
4  Divya       B     67     NaN  74.0          88
5  Imran       C     33    41.0  38.0          59
6  Sneha       A     81    77.0  85.0          90
7  Tarun       B     58    61.0  55.0          76
(8, 6)
```

CODE

```
df.info()
print(df.describe().round(1))
```

OUTPUT

```
<class 'pandas.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   name        8 non-null      str
 1   section     8 non-null      str
 2   maths       8 non-null      int64
 3   python      7 non-null      float64
 4   dbms        7 non-null      float64
 5   attendance  8 non-null      int64
dtypes: float64(2), int64(2), str(2)
memory usage: 516.0 bytes
       maths  python  dbms  attendance
count    8.0     7.0   7.0         8.0
mean    67.4    69.4  70.3        81.6
std     21.3    20.8  18.3        13.3
min     33.0    41.0  38.0        59.0
25%     54.8    56.5  62.5        74.0
50%     69.5    65.0  74.0        84.5
75%     82.8    84.5  82.0        91.0
max     95.0    98.0  91.0        97.0
```

OUTPUT WALKTHROUGH

- Empty cells appear as **NaN** (not a number). NaN is pandas' marker for 'missing'.
- **info()** lists the non-null count for each column. If it says 7 non-null out of 8 rows, one value is missing.
- **describe()** summarises count, mean, standard deviation, min, quartiles and max for numeric columns. Note that the *count* row is lower for python and dbms, which is another sign of missing values.

Selecting columns and rows

CODE

```
print(df[["name", "maths"]].head(3))
print(df.loc[2, "name"], df.iloc[0, 2])

view = df.assign(total=df["maths"] + df["python"])
print(view[["name", "maths", "python", "total"]].tail(5))
```

OUTPUT

```
    name  maths
0   Asha     88
1   Ravi     72
2  Meena     95
Meena 88
    name  maths  python  total
3  Karan     45    52.0   97.0
4  Divya     67     NaN    NaN
5  Imran     33    41.0   74.0
6  Sneha     81    77.0  158.0
7  Tarun     58    61.0  119.0
```

OUTPUT WALKTHROUGH

`df['maths']` returns one Series; `df[['name', 'maths']]` (double brackets) returns a smaller DataFrame. `.loc` selects by **label** (row 2, column 'name'), `.iloc` by **position** (row 0, column 2). The `total` column is computed for all rows at once, and Divya's missing python mark makes her total **NaN**: missing values spread through arithmetic.

Bridge from JSON: json_normalize

CODE

```
records = [
    {"name": "Asha", "marks": {"maths": 88, "python": 92}},
    {"name": "Ravi", "marks": {"maths": 72, "python": 65}},
]
print(pd.json_normalize(records))
```

OUTPUT

```
   name  marks.maths  marks.python
0  Asha           88            92
1  Ravi           72            65
```

PRACTICE CHALLENGES

1. Load `students.csv` and print the last 3 rows, the column names and the number of rows. *Hint: tail(3), df.columns, len(df).*

2. Add a column `best` holding the maximum of maths and python for each student. *Hint: df[['maths','python']].max(axis=1).*

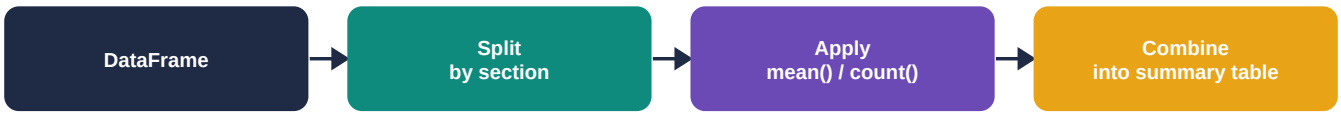
3. Save only the name and section columns to `roster.csv` without the index. *Hint: to_csv(..., index=False).*

4.2 Filtering, sorting and grouping

Three operations answer most business questions: **filter** (which rows?), **sort** (in what order?) and **group** (compared with what?). Filters use boolean conditions, combined with `&` (and), `|` (or) and `~` (not). **Each condition must be wrapped in parentheses.**

REAL-WORLD ANALOGY

Imagine a pile of exam papers. Filtering removes papers that do not match ('only section B'). Sorting puts the rest in order of marks. Grouping splits the pile into section bundles, computes each bundle's average, and writes the results on a summary sheet.



groupby is 'split, apply, combine'

CODE

```
both = df[(df["maths"] >= 70) & (df["python"] >= 70)]
print(both[["name", "maths", "python"]])

print(df.query("section == 'B' and attendance > 80")[["name", "attendance"]])

top3 = df.sort_values("maths", ascending=False).head(3)
print(top3[["name", "maths"]])
```

OUTPUT

```
    name  maths  python
0   Asha     88    92.0
2  Meena     95    98.0
6  Sneha     81    77.0
    name  attendance
1   Ravi          81
4  Divya          88
    name  maths
2  Meena     95
0   Asha     88
6  Sneha     81
```

OUTPUT WALKTHROUGH

Only Asha, Meena and Sneha score at least 70 in both. Note that Divya (python is NaN) is excluded because a comparison with NaN is always False. `query()` expresses the same idea as readable text. `sort_values` with `ascending=False` puts the highest first, and `.head(3)` keeps three.

Grouping and aggregation

CODE

```
print(df.groupby("section")[["maths", "python", "attendance"]].mean().round(1))

summary = df.groupby("section").agg(
    students=("name", "count"),
    avg_maths=("maths", "mean"),
    best_python=("python", "max"),
).round(1)
print(summary)
print(df["section"].value_counts())
```

OUTPUT

```
         maths  python  attendance
section
A         88.0    89.0        93.7
B         65.7    63.0        81.7
C         39.0    46.5        63.5
         students  avg_maths  best_python
section
A               3       88.0         98.0
B               3       65.7         65.0
C               2       39.0         52.0
section
A    3
B    3
C    2
Name: count, dtype: int64
```

OUTPUT WALKTHROUGH

The first table gives one row per section with average marks. Section A leads because Asha, Meena and Sneha are all in it. **Named aggregation** (`agg(new_name=(column, function))`) produces a tidy, self-documenting summary. `value_counts()` is a shortcut for counting how many rows belong to each category.

Goal	Code
Keep rows meeting a condition	`df[df['maths'] > 60]`
Either condition	`df[(df.maths > 90) | (df.python > 90)]`
Value is in a list	`df[df['section'].isin(['A', 'B'])]`

Goal	Code
Sort by two columns	`df.sort_values(['section', 'maths'], ascending=[True, False])`
Add a rank	`df['rank'] = df['maths'].rank(ascending=False)`

WATCH OUT

Writing `df[df.maths > 70 and df.python > 70]` raises a *ValueError*. Use `&` and parentheses instead, because Python's `and` cannot compare whole Series.

PRACTICE CHALLENGES

1. List students with attendance below 75 and maths below 50. *Hint: two conditions, both in parentheses.*

2. Show the average python mark for each section, highest section first. *Hint: groupby, mean, sort_values.*

3. Find the section with the most students who scored at least 80 in maths. *Hint: filter, then value_counts.*

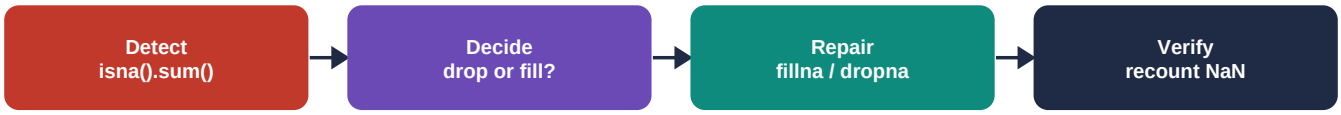
4. Create a `result` column with 'Pass' when both maths and python are at least 40 (use `np.where`). *Hint: combine conditions with &.*

4.3 Missing-data handling

Missing values are the most common data quality problem. The right fix depends on *why* the value is missing and how much data you can afford to lose. The workflow is always **detect, decide, repair, verify**.

REAL-WORLD ANALOGY

A class attendance register has blank boxes. You can (1) ignore students with any blank, (2) write the class average in the blank, (3) use that student's own section average, or (4) go back and ask the teacher. Each is defensible, but you must *choose deliberately* and note what you did.



Detect, decide, repair, verify

CODE

```
print(df.isna().sum())
print(df[df.isna().any(axis=1)][["name", "section", "python", "dbms"]])
print("rows if we drop:", len(df.dropna()), "of", len(df))
```

OUTPUT

```
name          0
section       0
maths         0
python        1
dbms          1
attendance    0
dtype: int64
    name section  python  dbms
3  Karan       C    52.0   NaN
4  Divya       B     NaN  74.0
rows if we drop: 6 of 8
```

Repairing: constant, statistic, group-aware

CODE

```
clean = df.copy()
clean["python"] = clean["python"].fillna(clean["python"].median())
clean["dbms"] = clean["dbms"].fillna(clean.groupby("section")["dbms"].transform("median"))

print(clean[["name", "section", "python", "dbms"]].loc[[3, 4]])
print("missing after:", clean.isna().sum().sum())

print("duplicates:", df.duplicated().sum())
clean.to_csv("students_clean.csv", index=False)
```

OUTPUT

```
     name section  python  dbms
3  Karan       C    52.0  38.0
4  Divya       B    65.0  74.0
missing after: 0
duplicates: 0
```

OUTPUT WALKTHROUGH

- Two problem rows: Karan (dbms missing) and Divya (python missing). Dropping them would lose 2 of 8 students, 25% of the data.
- **python** is filled with the column median (the typical value, not distorted by outliers).
- **dbms** uses `groupby(...).transform('median')`: each blank is replaced by the median of *its own section*. Karan is in section C, so he gets the median of section C's known dbms marks (Imran's 38).
- The final count of missing values is 0, which is the **verify** step. Always check.

Strategy	Code	Use when
Drop rows	`df.dropna()`	few rows affected, missing at random
Drop columns	`df.dropna(axis=1, thresh=...)`	column is mostly empty
Constant fill	`df['x'].fillna(0)`	blank truly means zero (e.g., no sales)
Mean / median fill	`fillna(df['x'].median())`	numeric, moderate gaps
Group-aware fill	`groupby(k)['x'].transform('median')`	groups differ noticeably
Forward fill	`df['x'].ffill()`	time series, last known value persists

WATCH OUT

Filling with the mean shrinks natural variation and can hide bias. Never fill the *target* column you want to predict without a clear reason, and record every cleaning decision in your project report: examiners and reviewers look for it.

PRACTICE CHALLENGES

1. Fill missing attendance with the section median and prove no NaN remains. *Hint: transform('median') then isna().sum().*

2. Create a column `data_quality` that says 'complete' or 'has gaps' for each student. *Hint: np.where(df.isna().any(axis=1), ...).*

3. Read the CSV again with `na_values=['N/A', '-', '']` and explain what it prevents. *Hint: text placeholders become NaN automatically.*

SECTION 5

Data Visualization

Turn tables of numbers into charts people understand in seconds

5.1 Matplotlib basics: bar and line charts

Matplotlib is Python's foundation plotting library. Every chart is a **Figure** (the canvas) holding one or more **Axes** (the individual plots). Use `fig, ax = plt.subplots()` and call methods on `ax`: this object-oriented style scales from one chart to a full dashboard.

REAL-WORLD ANALOGY

The Figure is a picture frame; each Axes is a photograph inside it. You can hang one photo or a gallery of four. Titles, labels and legends are the captions on each photo, and `savefig` takes a copy of the whole frame.



Every Matplotlib chart follows the same five steps

Question you are answering	Chart	Method
How do categories compare?	Bar chart	`ax.bar()`
How does a value change over time?	Line chart	`ax.plot()`
What share does each part have? (few parts)	Pie chart	`ax.pie()`
Are two numeric variables related?	Scatter plot	`ax.scatter()`
How is one variable distributed?	Histogram	`ax.hist()`

CODE

```
import matplotlib
matplotlib.use("Agg")            # lets scripts run without a screen; skip in Jupyter
import matplotlib.pyplot as plt

subjects  = ["Maths", "Python", "DBMS", "ML"]
avg_marks = [72.4, 78.9, 69.5, 74.2]
months    = ["Jul", "Aug", "Sep", "Oct", "Nov"]
sec_a     = [88, 90, 87, 92, 94]
sec_b     = [79, 82, 80, 85, 83]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 3.8))

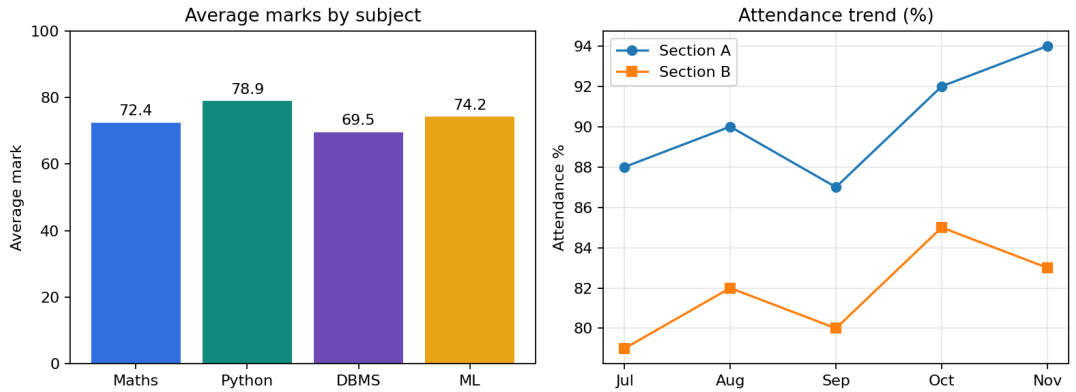
bars = ax1.bar(subjects, avg_marks, color=["#2F6FDE", "#0F8B7D", "#6C4AB6", "#E8A317"])
ax1.bar_label(bars, fmt="%.1f", padding=3)
ax1.set(title="Average marks by subject", ylabel="Average mark", ylim=(0, 100))

ax2.plot(months, sec_a, marker="o", label="Section A")
ax2.plot(months, sec_b, marker="s", label="Section B")
ax2.set(title="Attendance trend (%)", ylabel="Attendance %")
ax2.grid(alpha=0.3)
ax2.legend()

fig.tight_layout()
fig.savefig("bar_line.png", dpi=160)
plt.close(fig)
print("saved bar_line.png")
```

OUTPUT

```
saved bar_line.png
```



OUTPUT WALKTHROUGH

- **Left, bar chart:** categories on the x-axis, heights are averages. `bar_label` prints the value on each bar and `ylim=(0, 100)` keeps the axis honest by starting at zero.
- **Right, line chart:** two series share the axes, each with a marker and a legend label. Section A stays above B in every month.
- `ax.set(...)` sets several properties (title, labels, limits) in one call. `plt.close(fig)` frees memory, which matters when a script produces many charts.

PRACTICE CHALLENGES

1. Draw a horizontal bar chart of the four subject averages using `ax.barh`, sorted from highest to lowest. *Hint: sort the pairs first.*

2. Add a dashed horizontal line at the pass mark of 40 to a line chart using `ax.axhline`. *Hint: linestyle='--'.*

5.2 Pie charts and scatter plots

A **pie chart** shows parts of a whole and works only when there are few slices (five or fewer) that add up to something meaningful. A **scatter plot** places one dot per row to reveal relationships between two numeric columns. Adding a **trend line** makes the direction and strength of the relationship easier to read.

REAL-WORLD ANALOGY

A pie chart is slicing a pizza: everyone can see who got the bigger slice, but nobody can compare 12 thin slivers. A scatter plot is a map of a classroom: seat each student where their attendance and marks meet, and clusters or diagonal patterns show up immediately.

CODE

```
import numpy as np

grades = {"A": 9, "B": 14, "C": 11, "D": 4, "Fail": 2}
rng = np.random.default_rng(7)
attendance = rng.uniform(55, 100, 40)
avg_mark = 0.6 * attendance + rng.normal(15, 6, 40)

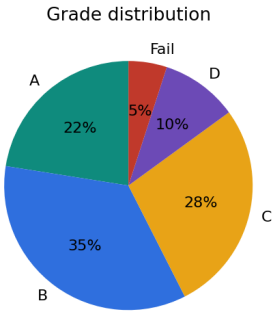
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 3.8))
ax1.pie(grades.values(), labels=grades.keys(), autopct="%1.0f%%", startangle=90,
        colors=["#0F8B7D", "#2F6FDE", "#E8A317", "#6C4AB6", "#C0392B"])
ax1.set_title("Grade distribution")

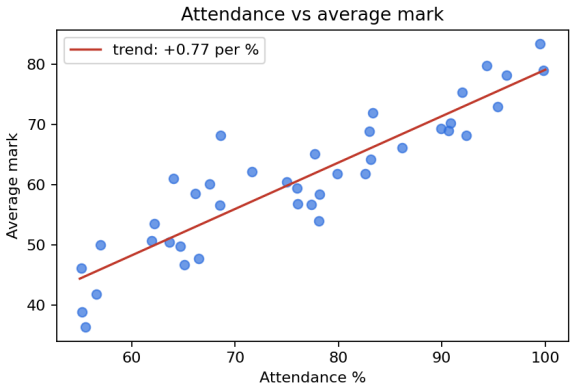
ax2.scatter(attendance, avg_mark, alpha=0.7, color="#2F6FDE")
slope, intercept = np.polyfit(attendance, avg_mark, 1)
xs = np.array([55, 100])
ax2.plot(xs, slope * xs + intercept, color="#C0392B", label=f"trend: +{slope:.2f} per %")
ax2.set(title="Attendance vs average mark", xlabel="Attendance %", ylabel="Average mark")
ax2.legend()

fig.tight_layout()
fig.savefig("pie_scatter.png", dpi=160)
plt.close(fig)
print("slope:", round(slope, 2))
```

OUTPUT

```
slope: 0.77
```





OUTPUT WALKTHROUGH

- **Pie:** `autopct` prints each slice's percentage and `startangle=90` starts the first slice at the top. Grades B and C together make up more than half of the class.
- **Scatter:** every dot is one student. The cloud rises left to right: *higher attendance goes with higher marks*.
- `np.polyfit(x, y, 1)` fits a straight line and returns its slope and intercept; the red line is drawn from those. The legend reports the slope: the fitted marks rise by that many points per attendance percentage point.

WATCH OUT

Correlation is not causation. A rising scatter does not prove attendance *causes* better marks; motivated students may do both. In your reports say 'is associated with' unless you have designed an experiment. Also avoid 3D pies, more than five slices, and truncated bar-chart axes that exaggerate differences.

Polish step	Code
Rotate long labels	`ax.tick_params(axis='x', rotation=45)`
Legend outside plot	`ax.legend(loc='upper left', bbox_to_anchor=(1, 1))`
Colour by category	`ax.scatter(x, y, c=df['section'].map(colors))`
Figure title over subplots	`fig.suptitle('Dashboard')`
Show (Jupyter or desktop)	`plt.show()`

PRACTICE CHALLENGES

1. Make a pie chart of how many students are in each section and explode the largest slice. *Hint: explode=(0.1, 0, 0).*

2. Draw a scatter plot of maths vs python marks from `students.csv` and label each dot with the student's name. *Hint: loop with ax.annotate.*

3. Create a 2x2 figure with a bar, line, pie and scatter chart, and save it as `dashboard.png`. *Hint: plt.subplots(2, 2); axes.flat.*

SECTION 6

APIs with Python

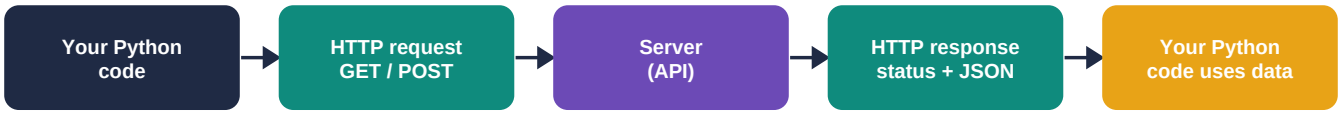
Let your programs talk to other programs over the internet

6.1 What is an API? GET and POST

An **API** (Application Programming Interface) is a menu of requests one program can make to another. Web APIs work over HTTP: your code sends a **request** to a URL and receives a **response**, usually in JSON. You never see how the server works inside; you only need to know the menu.

REAL-WORLD ANALOGY

A restaurant. You (the client) read the menu and tell the waiter (the API) what you want. The waiter carries the order to the kitchen (the server) and brings back your dish (the response). You do not walk into the kitchen. A GET is 'bring me the soup of the day'; a POST is 'here is my custom order, please prepare it'.



One API round trip

Method	Purpose	Sends data?	Typical use
GET	read / fetch data	in the URL (query parameters)	weather, list of posts, a user profile
POST	create / submit data	in the request body (JSON)	new order, submit a form, call an AI model
PUT / PATCH	update existing data	in the body	edit a profile
DELETE	remove data	usually none	delete a post

Your first GET request

CODE

```
import requests

url = "https://jsonplaceholder.typicode.com/posts/1"     # free practice API
response = requests.get(url, timeout=10)

print(response.status_code)
data = response.json()                  # JSON text -> Python dict
print(data["title"])
print(list(data.keys()))
```

SAMPLE OUTPUT

```
200
sunt aut facere repellat provident occaecati excepturi optio reprehenderit
['userId', 'id', 'title', 'body']
```

OUTPUT WALKTHROUGH

200 means success. `.json()` parses the response body into a normal Python dictionary, so everything you learned in Section 2.3 applies. `timeout=10` is essential: without it a stuck server can freeze your script forever. *(Outputs on this page are sample outputs because the network is needed to run them; run them yourself to confirm.)*

Query parameters and POST

CODE

```
# GET with filters: requests builds ...posts?userId=1 for you
r = requests.get("https://jsonplaceholder.typicode.com/posts",
                 params={"userId": 1}, timeout=10)
print(r.url)
print(len(r.json()), "posts")

# POST: send JSON in the body
payload = {"title": "Day 2", "body": "Learning APIs", "userId": 1}
r = requests.post("https://jsonplaceholder.typicode.com/posts", json=payload, timeout=10)
print(r.status_code)
print(r.json())
```

SAMPLE OUTPUT

```
https://jsonplaceholder.typicode.com/posts?userId=1
10 posts
201
{'title': 'Day 2', 'body': 'Learning APIs', 'userId': 1, 'id': 101}
```

OUTPUT WALKTHROUGH

The **params** dictionary becomes the query string after the question mark. For POST, `json=payload` converts your dict to JSON and sets the right header. **201 Created** tells you the server made a new resource and assigned it an id (101). This practice API pretends to save it but does not really store it.

PRACTICE CHALLENGES

1. GET `https://jsonplaceholder.typicode.com/users` and print the name and city of every user. *Hint: user['address']['city'].*

2. GET the posts of userId 3 and print how many have a title longer than 30 characters. *Hint: params + a comprehension.*

3. POST a new comment of your own to `/comments` and print the status code. *Hint: expect 201.*

6.2 JSON responses and status codes

Every response has two parts you must inspect: the **status code** (did it work?) and the **body** (what came back?). Professional scripts check the first before trusting the second.

Code	Meaning	What you should do
200 OK	request succeeded	use the data
201 Created	POST created a resource	read the new id from the body
204 No Content	success, nothing to return	nothing more to do
400 Bad Request	your request is malformed	check parameters and JSON
401 Unauthorized	missing or wrong API key	check the key and header
403 Forbidden	key valid, but not allowed	check your plan or permissions
404 Not Found	wrong URL or id	check the endpoint spelling
429 Too Many Requests	you hit the rate limit	wait, then retry more slowly
500 / 503	server-side problem	retry later; not your bug

REAL-WORLD ANALOGY

Status codes are like the reactions of a shop assistant. 2xx: 'here you go'. 4xx: 'you asked wrongly' (the customer's mistake). 5xx: 'sorry, our stockroom is on fire' (the shop's mistake). The first digit already tells you whose problem it is.



Decision flow after every request

Navigating a nested JSON response (runs offline)

CODE

```
import json
raw = '''{
  "latitude": 28.61, "longitude": 77.21,
  "current_weather": {"temperature": 31.4, "windspeed": 9.8, "is_day": 1},
  "hourly": {"time": ["10:00", "11:00", "12:00"], "temperature_2m": [30.1, 31.4, 32.6]}
}'''
data = json.loads(raw)

print("now:", data["current_weather"]["temperature"], "C")
hours = data["hourly"]["time"]
temps = data["hourly"]["temperature_2m"]
for t, temp in zip(hours, temps):
    print(t, temp)
print("hottest:", max(zip(temps, hours)))
print(data.get("humidity", "no humidity key"))
```

OUTPUT

```
now: 31.4 C
10:00 30.1
11:00 31.4
12:00 32.6
hottest: (32.6, '12:00')
no humidity key
```

OUTPUT WALKTHROUGH

Real API answers are nested: a dict containing dicts and parallel lists. You reach a value by following the keys (`data["current_weather"]["temperature"]`). Parallel lists pair up with `zip` from Section 1, and `max(zip(...))` finds the hottest hour. `.get()` protects you when a key is absent.

Defensive request function

CODE

```
def fetch_json(url, params=None):
    try:
        r = requests.get(url, params=params, timeout=10)
        r.raise_for_status()              # raises for 4xx / 5xx
        return r.json()
    except requests.exceptions.Timeout:
        print("Timed out, try again later")
    except requests.exceptions.HTTPError as err:
        print("Bad status:", err.response.status_code)
    except requests.exceptions.RequestException as err:
        print("Network problem:", err)
    return None

weather = fetch_json("https://api.open-meteo.com/v1/forecast",
                     {"latitude": 28.61, "longitude": 77.21, "current_weather": True})
if weather:
    print(weather["current_weather"]["temperature"])
```

PRO TIP

`raise_for_status()` converts error codes into exceptions, so a 404 can never sneak through as 'valid data'. Returning `None` on failure lets the caller decide what to do next.

PRACTICE CHALLENGES

1. Write `describe_status(code)` that returns 'success', 'your mistake' or 'server problem' using the first digit. *Hint: code // 100.*

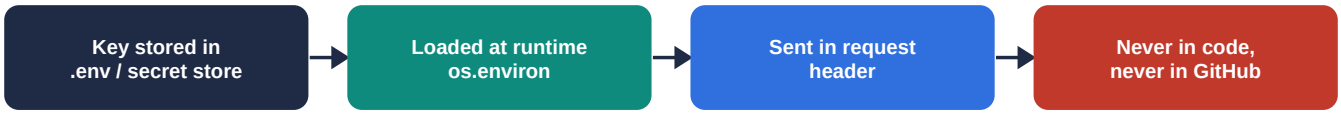
2. Given the weather JSON above, build a dict of hour -> temperature and print hours warmer than 31. *Hint: dict comprehension.*

3. Request a URL that does not exist (add /nope) and confirm your function prints 'Bad status: 404'. *Hint: fetch_json handles it.*

6.3 API keys and safe handling

Many APIs require an **API key**: a secret string that identifies you, controls your quota and may be billed. Anyone who has your key can spend your quota and act as you, so treat it like a password.

> **REAL-WORLD ANALOGY**
> An API key is the key card to your hotel room. It proves you are a registered guest and opens only your room. If you photograph it and post it online, strangers can walk in and charge meals to your bill. Cards can be deactivated and replaced, and you should do that the moment one leaks.



Keys travel from a private store to the request, never through your source code

The safe pattern

CODE

```
# .env  (a plain text file in your project folder; NEVER commit it)
WEATHER_API_KEY=abc123-your-real-key-here

# .gitignore  (tells git to skip secrets)
.env
venv/
__pycache__/
```

CODE

```
import os
import requests
from dotenv import load_dotenv

load_dotenv()                                   # reads .env into environment variables
API_KEY = os.environ.get("WEATHER_API_KEY")
if not API_KEY:
    raise SystemExit("Set WEATHER_API_KEY in your .env file first")

r = requests.get("https://api.example.com/data",
                 headers={"Authorization": f"Bearer {API_KEY}"},
                 timeout=10)
print(r.status_code)
```

Do	Do not
Keep keys in **.env** or environment variables	Paste keys directly into .py files or notebooks
Add .env to **.gitignore** before your first commit	Commit keys to GitHub, even in a private repo
Use a separate key for each project	Share one key with the whole class
Rotate (regenerate) a key the moment it leaks	Assume 'nobody will find it'
Check for the key at start-up and fail with a clear message	Print keys or full request headers in logs
Respect rate limits (sleep between calls)	Hammer an API in an endless loop

> **WATCH OUT**
> Bots scan public GitHub repositories continuously, and leaked keys are often abused within minutes. If a key ever reaches a repository, **revoke it first**; deleting the file afterwards does not help because git history keeps a copy.

> **PRO TIP**
> For your final-year project or hackathons, also add a `.env.example` file with fake values so teammates know which variables to set without seeing the real secrets.

PRACTICE CHALLENGES

1. Create a `.env` containing a fake key, load it, and print only the **first 4 characters** of it. *Hint: key[:4].*

2. Write a script that exits with a friendly message if the key is missing, and test it by renaming the variable. *Hint: SystemExit.*

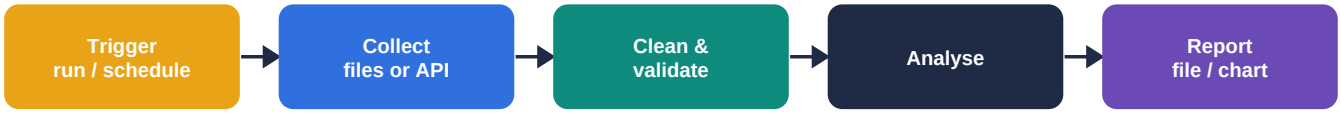
3. Explain in two sentences why git history matters when a secret is committed by accident. *Hint: think 'every commit is a saved copy'.*

SECTION 7

Automation

Let Python do the repetitive work: files, data, APIs and reports

A task is worth automating when you do it more than three times, it follows clear rules, and mistakes are costly. Automation is simply the earlier sections combined: **files in, cleaning and analysis in the middle, files or reports out.**



The shape of almost every automation script

7.1 File organization

REAL-WORLD ANALOGY

Imagine a tidy-up robot in a messy study room. It picks up each item, looks at what it is (the file extension), walks to the right shelf (the folder) and puts it there. If a shelf does not exist yet, it builds one.

CODE

```
from pathlib import Path
import shutil

src = Path("downloads")
src.mkdir(exist_ok=True)
files = ["notes.txt", "photo.jpg", "data.csv", "report.pdf",
         "song.mp3", "selfie.jpg", "budget.csv"]
for name in files:
    (src / name).write_text("demo")

folders = {".jpg": "Images", ".png": "Images", ".pdf": "Documents",
           ".txt": "Documents", ".csv": "Data", ".mp3": "Audio"}
moved = 0
for f in list(src.iterdir()):
    if f.is_file():
        target = src / folders.get(f.suffix.lower(), "Other")
        target.mkdir(exist_ok=True)
        shutil.move(str(f), target / f.name)
        moved += 1

print("moved", moved, "files")
for folder in sorted(p for p in src.iterdir() if p.is_dir()):
    print(folder.name, "->", sorted(x.name for x in folder.iterdir()))
```

OUTPUT

```
moved 7 files
Audio -> ['song.mp3']
Data -> ['budget.csv', 'data.csv']
Documents -> ['notes.txt', 'report.pdf']
Images -> ['photo.jpg', 'selfie.jpg']
```

OUTPUT WALKTHROUGH

- `Path` objects use `/` to join paths on any operating system. `.suffix` gives the extension.
- `folders.get(ext, 'Other')` maps each extension to a folder and sends unknown types to 'Other'.
- `list(src.iterdir())` takes a snapshot first, because creating folders while looping over the directory could confuse the loop.
- All 7 files are moved, and the last two lines print the tidy result: one line per new folder.

WATCH OUT

Test file-moving scripts on a **copy** of your folder first. Consider printing what *would* move (a 'dry run') before actually calling `shutil.move`, and never overwrite files without checking `target.exists()`.

PRACTICE CHALLENGES

1. Extend the organizer so files named like `invoice_2024.pdf` go to a 'Finance' folder. *Hint: check f.name.startswith('invoice').*

2. Add a dry-run flag that prints moves without performing them. *Hint: if not dry_run: shutil.move.*

7.2 CSV processing and data-cleaning automation

Organisations receive the same report every day, week or month as separate files. Instead of opening each in Excel, a short script can **find all files, combine them, clean them and summarise them** in seconds.



Batch CSV pipeline

CODE

```
import glob

Path("sales").mkdir(exist_ok=True)
months = {"jan": (["pen", "book"], [10, 4], [5, 120]),
          "feb": (["pen", "bag"], [12, 2], [5, 800]),
          "mar": (["book", "bag"], [6, 3], [120, 800])}
for m, (item, qty, price) in months.items():
    pd.DataFrame({"item": item, "qty": qty, "price": price}).to_csv(
        f"sales/{m}.csv", index=False)

frames = []
for path in sorted(glob.glob("sales/*.csv")):
    part = pd.read_csv(path)
    part["month"] = Path(path).stem            # file name without .csv
    frames.append(part)

sales = pd.concat(frames, ignore_index=True)
sales["revenue"] = sales["qty"] * sales["price"]
print(sales.groupby("month")["revenue"].sum())
print("best item:", sales.groupby("item")["revenue"].sum().idxmax())
```

OUTPUT

```
month
feb    1660
jan     530
mar    3120
Name: revenue, dtype: int64
best item: bag
```

OUTPUT WALKTHROUGH

Three monthly files are discovered with `glob`, each tagged with its month (taken from the file name), stacked with `pd.concat`, and summarised. Feb = 12x5 + 2x800 = 1660; Jan = 10x5 + 4x120 = 530; Mar = 6x120 + 3x800 = 3120. Adding a fourth file next month requires **no code change**: that is what automation buys you.

A reusable cleaning function

CODE

```
def clean_basic(df):
    df = df.copy()
    df.columns = df.columns.str.strip().str.lower()   # tidy headers
    df = df.drop_duplicates()
    for col in df.select_dtypes("number").columns:
        df[col] = df[col].fillna(df[col].median())
    return df

raw = pd.read_csv("students.csv")
tidy = clean_basic(raw)
print("missing before:", raw.isna().sum().sum(), "| after:", tidy.isna().sum().sum())
print(tidy.shape)
```

OUTPUT

```
missing before: 2 | after: 0
(8, 6)
```

PRO TIP

Put cleaning rules inside a **function**. You can then reuse it on every new file, test it on small samples, and improve it in one place. Scripts that clean data 'by hand' in a notebook cannot be trusted a month later.

PRACTICE CHALLENGES

1. Add a `source_file` column to the sales pipeline and export the combined table to `all_sales.csv`. *Hint: part['source_file'] = path.*

2. Extend `clean_basic` to also strip whitespace from every text column. *Hint: select_dtypes('object') or 'string'.*

7.3 API automation and report generation

API automation means calling an API on a schedule, saving what you receive, and producing an output a human can read. Three habits make such scripts dependable: **retry on temporary failure**, **cache results** so you do not waste quota, and **log what happened**.

REAL-WORLD ANALOGY

A newspaper delivery routine. Every morning the same steps happen: fetch the papers (API), sort them by street (clean), deliver (report). If a road is closed you try again later (retry), and you keep a list of what was delivered (log).



An automated API-to-report pipeline

CODE

```
import time, json, requests
from pathlib import Path

def get_with_retry(url, params=None, tries=3):
    for attempt in range(1, tries + 1):
        try:
            r = requests.get(url, params=params, timeout=10)
            if r.status_code == 429 or r.status_code >= 500:
                raise requests.exceptions.RequestException(f"status {r.status_code}")
            r.raise_for_status()
            return r.json()
        except requests.exceptions.RequestException as err:
            wait = 2 ** attempt                        # 2s, 4s, 8s: exponential backoff
            print(f"attempt {attempt} failed ({err}); waiting {wait}s")
            time.sleep(wait)
    return None

cache = Path("posts_cache.json")
if cache.exists():
    posts = json.loads(cache.read_text())              # reuse yesterday's download
else:
    posts = get_with_retry("https://jsonplaceholder.typicode.com/posts")
    cache.write_text(json.dumps(posts))
```

OUTPUT WALKTHROUGH

Backoff doubles the wait after each failure so you do not hammer a struggling server. Only temporary problems (429 and 5xx) trigger a retry; a 404 is a bug in your request and retrying would not fix it. The **cache** file means a second run is instant and uses no quota.

Generating a report (runs offline)

CODE

```
clean = pd.read_csv("students_clean.csv")
clean["avg"] = clean[["maths", "python", "dbms"]].mean(axis=1).round(1)
top = clean.loc[clean["avg"].idxmax()]
risk = ", ".join(clean.loc[clean["avg"] < 50, "name"])

lines = [
    "STUDENT PERFORMANCE REPORT",
    "=" * 30,
    f"Students      : {len(clean)}",
    f"Class average : {clean['avg'].mean():.1f}",
    f"Top student   : {top['name']} ({top['avg']})",
    f"At risk (<50) : {risk}",
]
Path("report.txt").write_text("\n".join(lines))
print(Path("report.txt").read_text())
```

OUTPUT

```
STUDENT PERFORMANCE REPORT
==============================
Students      : 8
Class average : 67.5
Top student   : Meena (94.7)
At risk (<50) : Karan, Imran
```

Way to run automatically	Where	Example
cron	Linux / macOS	`0 8 * * 1-5 python report.py` (weekdays 08:00)
Task Scheduler	Windows	trigger daily, action: run python.exe with your script
schedule library	any (script keeps running)	`schedule.every().day.at('08:00').do(job)`
GitHub Actions / cloud	servers	run on a timer without your laptop

PRACTICE CHALLENGES

1. Add a chart (bar of averages per student) saved as `report.png` to the report script. *Hint: reuse Section 5 code.*

2. Make the report include today's date using `datetime.date.today()` and save it as `report_YYYY-MM-DD.txt`. *Hint: f-string filename.*

3. Write a script that calls the weather API for three cities and saves the temperatures to a CSV. *Hint: loop over a dict of coordinates.*

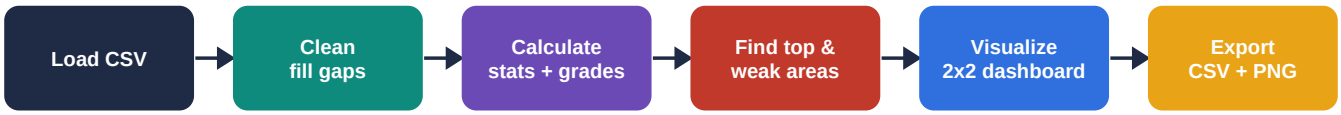
SECTION 8

Hands-On Projects

Combine everything: analyse, visualize and clean real-looking data

Project 1: AI Student Analytics Dashboard

Goal: load a student CSV, clean it, calculate statistics, identify top and weak areas, and visualize the results in one dashboard. This is the kind of mini-product a college or coaching centre would actually use.



Project 1 pipeline

Step 1: generate the dataset

If you do not have a real file, this script creates a realistic 40-student dataset with a few deliberate gaps. The random seed makes your file identical to the one used for the outputs in this workbook.

CODE

```
import numpy as np, pandas as pd
rng = np.random.default_rng(42)
names = ["Aarav", "Diya", "Kabir", "Meera", "Rohan", "Ananya", "Vihaan", "Isha",
         "Arjun", "Saanvi", "Reyansh", "Kavya", "Advait", "Nisha", "Ishaan", "Pooja",
         "Dev", "Riya", "Yash", "Tanvi", "Kunal", "Sneha", "Harsh", "Neha", "Varun",
         "Priya", "Manav", "Aditi", "Rahul", "Simran", "Nikhil", "Anjali", "Tejas",
         "Bhavna", "Sahil", "Divya", "Pranav", "Kiran", "Om", "Lakshmi"]
n = len(names)
skill = rng.normal(0, 1, n)                  # hidden 'ability' links marks together
data = pd.DataFrame({
    "student_id": range(101, 101 + n),
    "name": names,
    "section": rng.choice(["A", "B", "C"], n),
    "attendance": (80 + 6 * skill + rng.normal(0, 8, n)).clip(45, 100).round(0),
})
specs = [("maths", 68, 14), ("python", 74, 13), ("dbms", 66, 15), ("ml", 62, 16)]
for subject, mean, sd in specs:
    data[subject] = (mean + 9 * skill + rng.normal(0, sd * 0.6, n)).clip(15, 100).round(0)
data.loc[[4, 17, 29], "ml"] = np.nan          # deliberate gaps
data.loc[[9], "python"] = np.nan
data.loc[[22], "attendance"] = np.nan
data.to_csv("class_students.csv", index=False)
print(data.shape, "| missing cells:", int(data.isna().sum().sum()))
```

OUTPUT

```
(40, 8) | missing cells: 5
```

Step 2: load, clean and add calculated columns

CODE

```
df = pd.read_csv("class_students.csv")
subjects = ["maths", "python", "dbms", "ml"]
cols = subjects + ["attendance"]

print(df[cols].isna().sum().to_dict())
for col in cols:                                    # repair with the column median
    df[col] = df[col].fillna(df[col].median())

df["total"] = df[subjects].sum(axis=1)
df["average"] = df[subjects].mean(axis=1).round(1)
df["grade"] = pd.cut(df["average"], bins=[0, 40, 50, 60, 75, 101],
                     labels=["Fail", "D", "C", "B", "A"], right=False)
print("missing after:", int(df[cols].isna().sum().sum()))
print(df[["name", "section", "total", "average", "grade"]].head(4).to_string(index=False))
```

OUTPUT

```
{'maths': 0, 'python': 1, 'dbms': 0, 'ml': 3, 'attendance': 1}
missing after: 0
 name section  total  average grade
Aarav       C  297.0     74.2     B
 Diya       B  243.0     60.8     B
Kabir       A  308.0     77.0     A
Meera       C  311.0     77.8     A
```

OUTPUT WALKTHROUGH

Missing values are reported *before* repair (so you can quote them in your report) and re-counted *after*. `pd.cut` converts the numeric average into grade bands: below 40 is Fail, 40-49 is D, 50-59 is C, 60-74 is B and 75 or more is A. The `.to_string(index=False)` call prints a tidy table without the row numbers.

Step 3: statistics, top performers and weak areas

CODE

```
print(df[subjects].describe().loc[["mean", "std", "min", "max"]].round(1))
print("class average:", round(df["average"].mean(), 1))
print(df["grade"].value_counts().reindex(["A", "B", "C", "D", "Fail"]).to_dict())

top5 = df.nlargest(5, "average")[["name", "section", "average", "attendance"]]
print(top5.to_string(index=False))

at_risk = df[(df["average"] < 50) | (df["attendance"] < 70)]
print("at risk:", len(at_risk), "students ->", ", ".join(at_risk["name"]))

subject_avg = df[subjects].mean().round(1)
fail_rate = ((df[subjects] < 40).mean() * 100).round(1)
print(pd.DataFrame({"avg": subject_avg, "fail_%": fail_rate}))
print("weakest subject:", subject_avg.idxmin(), "| strongest:", subject_avg.idxmax())
print(df.groupby("section")["average"].mean().round(1).to_dict())
```

OUTPUT

```
      maths  python  dbms     ml
mean   67.3    74.3  67.0   61.4
std    11.2    10.4  10.7   12.4
min    47.0    40.0  45.0   40.0
max    87.0    92.0  88.0  100.0
class average: 67.5
{'A': 6, 'B': 28, 'C': 5, 'D': 1, 'Fail': 0}
   name section  average  attendance
 Nikhil       A     87.8        97.0
  Meera       C     77.8        83.0
  Sahil       B     77.8        80.0
Lakshmi       A     77.5        76.0
  Kabir       A     77.0        71.0
at risk: 4 students -> Rohan, Pranav, Kiran, Om
         avg  fail_%
maths   67.3     0.0
python  74.3     0.0
dbms    67.0     0.0
ml      61.4     0.0
weakest subject: ml | strongest: python
{'A': 68.1, 'B': 66.8, 'C': 67.5}
```

OUTPUT WALKTHROUGH

- **Top areas:** `nlargest` gives the five best students; `idxmax` names the strongest subject.
- **Weak areas:** at-risk students are those with an average below 50 *or* attendance below 70; `idxmin` names the weakest subject; the fail-rate table shows what percentage scored below 40 in each subject.
- **Segments:** the last line compares average marks across sections so you can tell whether the problem is one class or the whole cohort.

Step 4: the dashboard

CODE

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 2, figsize=(11, 8))
colors = ["#2F6FDE", "#0F8B7D", "#6C4AB6", "#E8A317"]

b = axes[0, 0].bar([s.title() for s in subjects], subject_avg.values, color=colors)
axes[0, 0].bar_label(b, fmt="%.1f", padding=2)
axes[0, 0].set(title="Average marks by subject", ylim=(0, 100))

counts = df["grade"].value_counts().reindex(["A", "B", "C", "D", "Fail"])
axes[0, 1].pie(counts, labels=counts.index, autopct="%1.0f%%", startangle=90,
               colors=["#0F8B7D", "#2F6FDE", "#E8A317", "#6C4AB6", "#C0392B"])
axes[0, 1].set_title("Grade distribution")

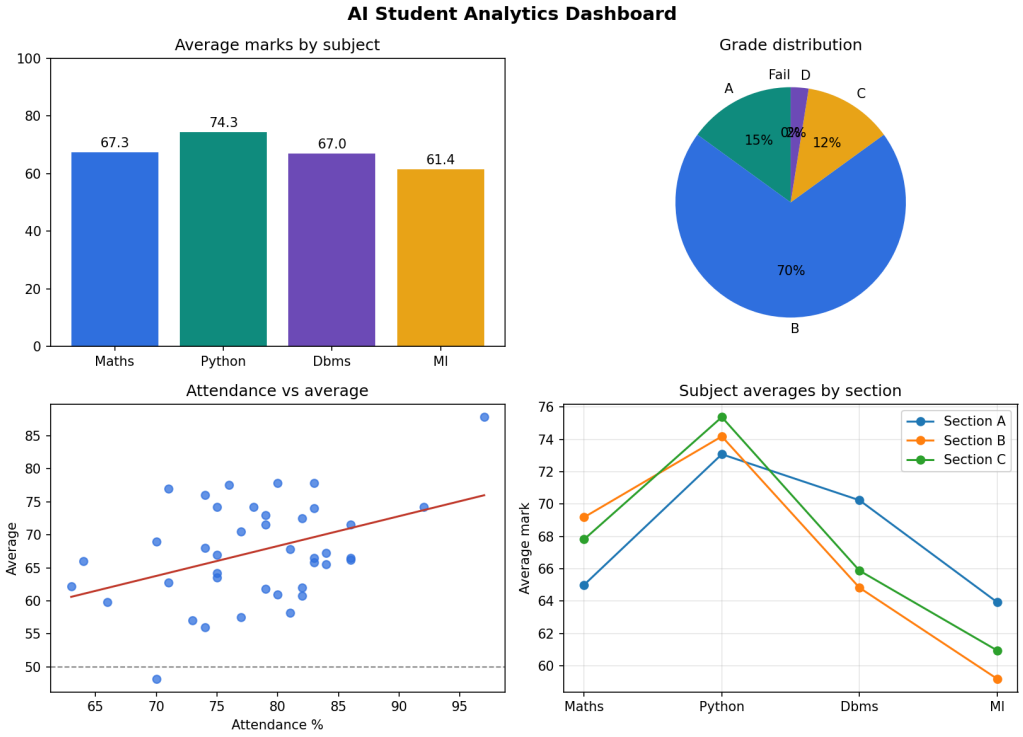
axes[1, 0].scatter(df["attendance"], df["average"], alpha=0.75, color="#2F6FDE")
m, k = np.polyfit(df["attendance"], df["average"], 1)
xs = np.array([df["attendance"].min(), df["attendance"].max()])
axes[1, 0].plot(xs, m * xs + k, color="#C0392B")
axes[1, 0].axhline(50, color="grey", linestyle="--", linewidth=1)
axes[1, 0].set(title="Attendance vs average", xlabel="Attendance %", ylabel="Average")

for sec, grp in df.groupby("section"):
    axes[1, 1].plot([s.title() for s in subjects], grp[subjects].mean(),
                    marker="o", label="Section " + sec)
axes[1, 1].set(title="Subject averages by section", ylabel="Average mark")
axes[1, 1].legend()
axes[1, 1].grid(alpha=0.3)

fig.suptitle("AI Student Analytics Dashboard", fontsize=14, fontweight="bold")
fig.tight_layout()
fig.savefig("dashboard.png", dpi=150)
at_risk.to_csv("at_risk_students.csv", index=False)
print("saved dashboard.png and at_risk_students.csv")
```

OUTPUT

```
saved dashboard.png and at_risk_students.csv
```



OUTPUT WALKTHROUGH

The dashboard answers four questions at a glance: *which subject is weakest* (bar), *how are grades distributed* (pie), *does attendance matter* (scatter with trend and a dashed pass line at 50), and *do sections differ* (lines). The at-risk CSV is the actionable output a mentor could receive.

EXTENSION CHALLENGES

- **Level 1:** add a `rank` column and export a ranked `leaderboard.csv`.
- **Level 2:** add a histogram of averages and a horizontal line for the class mean.
- **Level 3:** wrap Steps 2 to 4 in a function `build_dashboard(csv_path)` and run it on a second CSV.
- **Level 4:** write a text report (Section 7.3) that lists the weakest subject, top 3 students and at-risk names.
- **Level 5 (portfolio):** show the dashboard in a web page using Streamlit: `pip install streamlit`, then `st.pyplot(fig)`.

Project 2: Messy Data Challenge

Goal: clean a deliberately dirty dataset containing **missing values, duplicates, incorrect formats and invalid marks**. Real datasets arrive like this. Try to write the cleaning code yourself from the checklist *before* reading the solution.

The dirty file

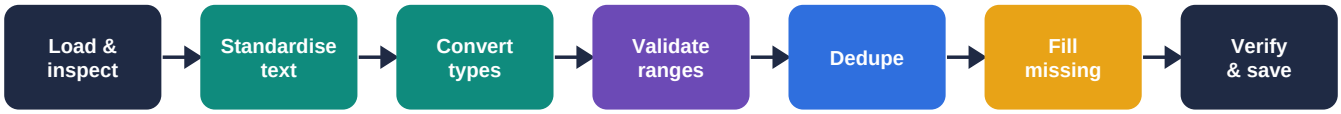
CODE

```
messy = '''id,name,section,maths,python,joined
1, Asha ,a,88,92,2022-06-15
2,ravi,B,72,65,15/06/2022
3,MEENA,A,95,98,2022-06-16
3,MEENA,A,95,98,2022-06-16
4,Karan,c,45,N/A,2022-06-17
5,Divya,B,seventy,74,17-06-2022
6,Imran,C,-5,41,2022-06-18
7,Sneha,a,181,77,2022-06-18
8,Tarun,B,58,,2022/06/19
9,ravi,B,72,65,15/06/2022'''
Path("messy_students.csv").write_text(messy)
raw = pd.read_csv("messy_students.csv")
print(raw.to_string())
```

OUTPUT

```
   id    name section    maths  python      joined
0   1   Asha        a       88    92.0  2022-06-15
1   2    ravi       B       72    65.0  15/06/2022
2   3   MEENA       A       95    98.0  2022-06-16
3   3   MEENA       A       95    98.0  2022-06-16
4   4   Karan       c       45     NaN  2022-06-17
5   5   Divya       B  seventy    74.0  17-06-2022
6   6   Imran       C       -5    41.0  2022-06-18
7   7   Sneha       a      181    77.0  2022-06-18
8   8   Tarun       B       58     NaN  2022/06/19
9   9    ravi       B       72    65.0  15/06/2022
```

Problem	Where	How to detect	Fix
Extra spaces, mixed case	name, section	`df.name.unique()`	`.str.strip().str.title()`
Exact duplicate row	id 3	`df.duplicated().sum()`	`drop_duplicates()`
Hidden duplicate (same person, different id)	ravi, id 2 and 9	normalise text first, then check	`drop_duplicates(subset=[...])`
Text in numeric column	'seventy', 'N/A'	`df.dtypes` shows object	`pd.to_numeric(errors='coerce')`
Invalid marks	-5 and 181	`df[(df.maths < 0) | (df.maths > 100)]`	set to NaN, then repair
Inconsistent date formats	joined	eyeball or parse errors	`pd.to_datetime(format='mixed', dayfirst=True)`
Missing values	python, maths	`df.isna().sum()`	median fill (documented)



Order matters: standardise text before looking for duplicates, and validate ranges before filling gaps

> **REAL-WORLD ANALOGY**
> Cleaning data is washing vegetables before cooking. You wash off the dirt (spaces, case), discard rotten pieces (invalid values), remove doubles (duplicates), cut everything to a consistent size (formats), and only then start cooking (analysis). Cooking first and washing later ruins the dish.

A worked solution

CODE

```
df = raw.copy()
df["name"] = df["name"].str.strip().str.title()
df["section"] = df["section"].str.strip().str.upper()

for col in ["maths", "python"]:
    df[col] = pd.to_numeric(df[col], errors="coerce")       # text -> NaN
    df.loc[(df[col] < 0) | (df[col] > 100), col] = np.nan   # invalid -> NaN
df["joined"] = pd.to_datetime(df["joined"], format="mixed", dayfirst=True)

before = len(df)
df = df.drop_duplicates(subset=["name", "section"], keep="first")
print("duplicates removed:", before - len(df))
print("gaps to repair:", df[["maths", "python"]].isna().sum().to_dict())

for col in ["maths", "python"]:
    df[col] = df[col].fillna(df[col].median())
df = df.reset_index(drop=True)
print(df.to_string())
```

OUTPUT

```
duplicates removed: 2
gaps to repair: {'maths': 3, 'python': 2}
   id   name section  maths  python     joined
0   1   Asha       A   88.0    92.0 2022-06-15
1   2   Ravi       B   72.0    65.0 2022-06-15
2   3  Meena       A   95.0    98.0 2022-06-16
3   4  Karan       C   45.0    75.5 2022-06-17
4   5  Divya       B   72.0    74.0 2022-06-17
5   6  Imran       C   72.0    41.0 2022-06-18
6   7  Sneha       A   72.0    77.0 2022-06-18
7   8  Tarun       B   58.0    75.5 2022-06-19
```

CODE

```
assert df["maths"].between(0, 100).all() and df["python"].between(0, 100).all()
assert not df.duplicated(subset=["name", "section"]).any()
assert df.isna().sum().sum() == 0
print("all checks passed |", df.shape[0], "clean rows")
df.to_csv("messy_students_clean.csv", index=False)
```

OUTPUT

```
all checks passed | 8 clean rows
```

OUTPUT WALKTHROUGH

- Both duplicates disappear: id 3 is an exact copy, and id 9 is Ravi again under a different id (found only *after* the names were normalised).
- 'seventy', 'N/A', -5 and 181 all became NaN, then were repaired with the median of the valid marks. Six gaps in total were filled.
- All four date styles now share one proper datetime type, so you can sort by date or compute durations.
- The three `assert` lines are **tests**: if a later change breaks cleaning, the script stops loudly instead of producing bad reports.

WATCH OUT

Filling 'seventy' with the median is a shortcut. A careful analyst would recover the obvious value (70) manually and record the decision. Always keep the **raw file untouched** and write cleaned data to a new file, so every step is reproducible.

EXTENSION CHALLENGES

- **Bonus 1:** map spelled-out numbers ('seventy') to digits with a small dictionary before `to_numeric`.
- **Bonus 2:** produce a `data_quality_report.txt` counting how many cells were fixed per category.
- **Bonus 3:** read the file with `na_values=['N/A']` and `skipinitialspace=True` and remove the lines of code these make unnecessary.
- **Bonus 4:** turn the whole solution into `clean_students(path_in, path_out)` and run it on a new dirty file you design yourself.

SECTION 9

Reference

Cheat sheet, self-check quiz and answer key

Cheat sheet

Topic	Essential syntax
List comprehension	`[f(x) for x in xs if cond]`
Dict / set comprehension	`{k: v for k, v in pairs}` | `{x for x in xs}`
Built-ins	`map(f, xs)` `filter(f, xs)` `zip(a, b)` `enumerate(xs, start=1)`
Text files	`with open(p, 'w') as f: f.write(text)`
CSV	`csv.DictReader(f)` | `csv.DictWriter(f, fieldnames)` (open with newline='')
JSON	`json.load(f)` `json.dump(obj, f, indent=2)` `json.loads(s)` `json.dumps(obj)`
NumPy	`np.array` `a.mean(axis=0)` `a[a > 0]` `np.where(cond, x, y)`
pandas load / inspect	`pd.read_csv` `df.head()` `df.info()` `df.describe()`
pandas filter / sort	`df[(a) & (b)]` `df.sort_values(col)` `df.nlargest(n, col)`
pandas group	`df.groupby(k)[cols].mean()` `df.agg(name=(col, 'mean'))`
Missing / cleaning	`isna().sum()` `fillna` `dropna` `drop_duplicates` `to_numeric(errors='coerce')`
Matplotlib	`fig, ax = plt.subplots()` `ax.bar / plot / pie / scatter` `fig.savefig`
requests	`requests.get(url, params=..., timeout=10)` `requests.post(url, json=...)` `r.raise_for_status()`
Secrets	`load_dotenv()` `os.environ.get('KEY')` and .gitignore your .env
Automation	`Path.iterdir()` `shutil.move` `glob.glob('*.csv')` `pd.concat(frames)`

Self-check quiz (answers follow)

1. What does `[x * 2 for x in range(4) if x % 2 == 0]` produce?
2. Why does `zip(a, b)` sometimes silently lose data?
3. Which open mode erases an existing file: `'r'`, `'w'` or `'a'`?
4. After `csv.DictReader`, what type does the value '88' have?
5. What does `json.dumps` return, and how is it different from `json.dump`?
6. For a 2-D array, which axis gives one result per *row*?
7. Why does `df[df.a > 1 and df.b > 1]` fail, and what is the fix?
8. Name two ways to handle missing values and one risk of each.
9. Which status-code family means the client made a mistake?
10. Why should an API key never appear in your source code?

Answer key

1. `[0, 4]` (x takes 0, 2; doubled gives 0 and 4).
2. It stops at the shortest input. Check lengths or use `strict=True`.
3. `'w'` truncates the file on open.
4. A string, `str`. Convert with `int()` or `float()`.
5. A JSON-formatted **string**; `dump` writes to a file object instead.
6. `axis=1` (collapse the columns).
7. `and` cannot compare whole Series. Use `(df.a > 1) & (df.b > 1)` with parentheses.
8. Dropping rows loses data; filling with a mean hides natural variation and can bias results.
9. 4xx (for example 400, 401, 404).
10. Anyone with the file (or the git history) can use and abuse the key. Use .env and .gitignore.

KEY IDEA

Congratulations, you now have the toolkit behind most real data projects: read data, clean it, analyse it, show it, and automate it. **Next step:** pick a dataset you care about (cricket, movies, your own college results), run the same five-stage pipeline, and publish the code with a README on GitHub.